

Industrial AI Twin

Technical Architecture Document

Edge-Based Predictive Maintenance System for Machine Fault Detection

Document Type	Technical Architecture
Project	Industrial AI Twin
Module Context	Edge and Embedded Intelligence
Prepared For	Professor / Supervisor Review
Prepared By	Abdulrehman Mohamedrafiq Vohra
Version	1.0

Architecture focus: hardware, embedded sensing, BLE data transport, Raspberry Pi edge AI processing, dashboard services, fault logic, reliability and future expansion.

Document Control

Version	Date	Section	Description
1.0	15 June 2026	All	Initial technical architecture document for professor review.

1. Executive Technical Summary

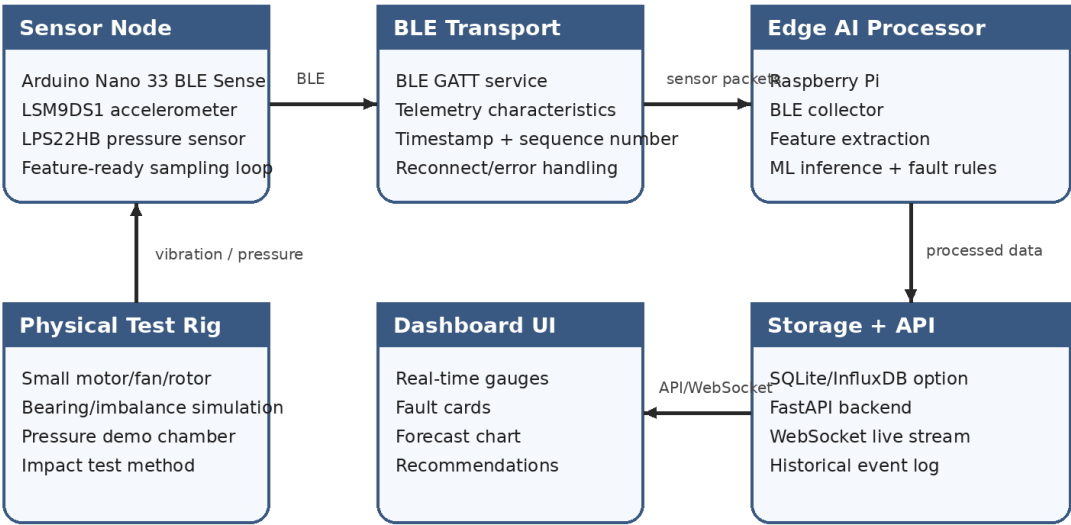
Industrial AI Twin is proposed as an edge-based predictive maintenance prototype that creates a live virtual health representation of a small industrial machine or controlled test rig. The system uses an Arduino Nano 33 BLE Sense as the embedded sensing node, BLE as the low-power telemetry channel, and a Raspberry Pi as the edge AI processor. The dashboard displays current condition, fault states, short-term forecasts and maintenance recommendations.

The architecture is intentionally hybrid. Raw sensing is performed on the microcontroller, while heavier feature extraction, model inference, data storage and dashboard delivery run on the Raspberry Pi. This is suitable for a coursework prototype because it demonstrates embedded sensing and edge intelligence without pretending that a small microcontroller is a full industrial analytics server.

Key design challenge: the built-in LPS22HB is a barometric absolute pressure sensor, not a direct industrial pipe pressure transducer. The architecture therefore treats pressure anomaly detection as either a controlled demo scenario or recommends an external pressure transducer for the stronger implementation.

2. High-Level System Architecture

Industrial AI Twin - Technical Architecture



Design principle: perform time-critical sensing near the machine, run heavier inference at the edge, and present explainable fault states to the operator

Figure 1. Proposed end-to-end architecture from physical machine sensing to dashboard-level fault intelligence.

3. Architecture Goals and Design Principles

Principle	Architecture Implication
Edge-first operation	The system should continue sensing, processing and displaying local machine state even without cloud connectivity.
Explainable fault logic	Fault states should be based on visible features such as vibration trend, kurtosis, RMS, pressure deviation and spike duration.
Prototype honesty	The document separates proven prototype behaviour from future industrial expansion so the system remains academically defensible.
Low-latency monitoring	Critical dashboard states should update within seconds, especially impact and vibration alarms.
Modular expansion	Additional fault modes such as imbalance, misalignment and looseness should be addable without redesigning the whole system.
Safe demonstration	The test rig must be low-voltage, mechanically guarded and safe for classroom demonstration.

4. Hardware Architecture and Bill of Materials

The hardware is divided into mandatory prototype hardware, recommended reliability hardware, and optional enhancement hardware. This separation helps the professor see what is essential for the coursework and what is an improvement path.

Hardware	Priority	Role	Use in Project
Arduino Nano 33 BLE Sense	Mandatory	Embedded sensor node	Reads vibration and pressure-related telemetry close to the machine. Provides BLE connectivity and onboard sensors in a compact form factor.
LSM9DS1 triaxial accelerometer / IMU	Mandatory if using original board	Vibration sensing	Measures acceleration on X/Y/Z axes. Used to calculate resultant vibration, RMS, peaks, spikes, trend and kurtosis. Note: confirm board revision because Rev2 sensor set may differ.
LPS22HB barometric pressure sensor	Mandatory in professor proposal	Pressure/environment signal	Measures absolute barometric pressure. Suitable for controlled pressure demo or environmental baseline deviation, but not a direct industrial pipe pressure sensor without a physical pressure

			setup.
Raspberry Pi 4 or Raspberry Pi 5	Mandatory	Edge AI gateway and local server	Receives BLE data, performs feature extraction, runs ML/analytics modules, stores data and serves the dashboard.
MicroSD card, 32 GB or higher	Mandatory	Operating system and local storage	Stores Raspberry Pi OS, Python services, dashboard files, logs and local database.
5V/3A USB-C power supply for Raspberry Pi	Mandatory	Stable edge power	Prevents undervoltage resets during dashboard and BLE operation.
USB cable / battery pack for Arduino	Mandatory	Sensor node power	Powers the Arduino during testing. Battery operation can demonstrate wireless edge deployment.
Small DC motor, fan, pump or rotating test rig	Mandatory for meaningful demo	Physical machine simulator	Provides a controllable machine source for vibration events and trend simulation.
Mounting bracket, screws, adhesive pad or 3D printed holder	Mandatory	Sensor coupling	Firmly attaches the sensor node to the machine/test rig. Poor mounting creates noisy and misleading vibration readings.
Protective enclosure for electronics	Recommended	Safety and robustness	Protects boards from vibration, accidental contact and wiring damage.
External industrial pressure transducer, 0-1 bar or suitable range	Recommended upgrade	Credible leak/blockage pressure measurement	Replaces or supplements the LPS22HB when demonstrating real pressure-based leak/blockage detection.
ADS1115 ADC or compatible ADC module	Conditional	Analog sensor interface	Needed if the selected pressure transducer outputs analog voltage and the Raspberry Pi/Arduino needs precision analog reading.
Breadboard, jumper wires, resistors, connectors	Recommended	Prototyping interconnect	Used for safe wiring, test rig integration and optional external sensors.
Heat sink or small fan for Raspberry Pi	Recommended	Thermal reliability	Improves reliability during long-running dashboard/inference sessions.
Portable HDMI display or laptop browser	Optional	Dashboard display	Displays dashboard locally for demonstration.
LED/buzzer indicator	Optional	Local fault feedback	Provides visible/audible local indication for critical states without relying only on the browser UI.
Camera module	Future optional	Visual digital twin extension	Could support visual inspection or live machine view, but not required for the first version.

5. Hardware Selection Rationale

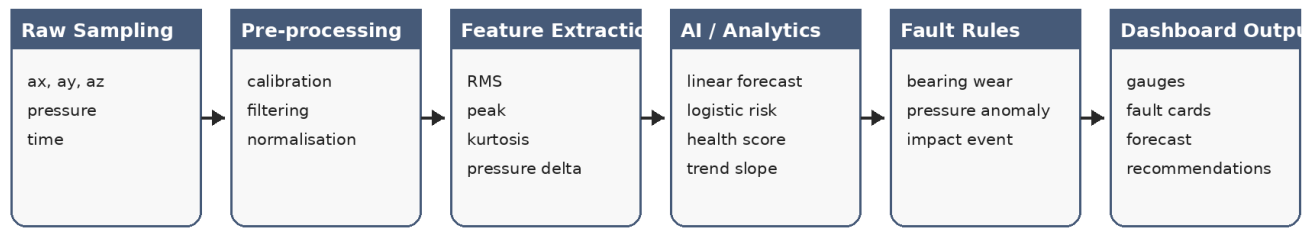
Decision	Reason
Arduino Nano 33 BLE Sense	Chosen because it integrates BLE and multiple sensors in a small embedded board, reducing wiring complexity and making it suitable for a clean prototype.
Raspberry Pi edge processor	Chosen because it can run Python, BLE libraries, ML models, database services and a web dashboard locally. This creates a realistic edge gateway architecture.
BLE communication	Chosen because it demonstrates low-power wireless telemetry between an embedded node and an edge processor. It also avoids USB-only demos that look less industrial.
Small machine/test rig	Needed because predictive maintenance cannot be convincingly demonstrated from static sensor readings. The project needs a physical vibration source.
External pressure transducer option	Included because the built-in pressure sensor is not ideal for real leak/blockage detection. Adding this option protects the architecture from a credibility challenge.

6. Software Architecture

Layer	Suggested Technology	Responsibility
Arduino firmware	Arduino C++ / ArduinoBLE / sensor libraries	Samples accelerometer and pressure data, packages readings and exposes BLE GATT characteristics.
BLE collector service	Python on Raspberry Pi, e.g. bleak	Connects to Arduino, subscribes to telemetry notifications, validates packet sequence and handles reconnection.
Feature extraction service	Python, NumPy/SciPy style calculations	Converts raw acceleration into resultant vibration, RMS, peak, standard deviation, kurtosis, moving average, trend slope and pressure deviation.
AI inference service	Python, scikit-learn or lightweight custom models	Runs linear regression forecasting and logistic regression failure probability.
Fault rule engine	Python business logic	Combines thresholds, model outputs and persistence rules to determine active fault status.
Local data storage	SQLite for simple version; InfluxDB optional	Stores raw readings, extracted features, predictions, health scores and event history.
Backend API	FastAPI recommended	Serves current status, historical readings, forecasts, event logs and recommendations to the dashboard.
Real-time streaming	WebSocket or Server-Sent Events	Pushes live telemetry and fault state changes to the browser dashboard.
Dashboard frontend	React/Vite or simple HTML + Chart.js/ECharts	Displays gauges, trend chart, fault cards, forecast scenarios and recommendation panel.
Service management	systemd on Raspberry Pi	Keeps BLE collector/API/dashboard service running after reboot.

7. Data and AI Processing Pipeline

Data and AI Processing Pipeline



Fault logic is deliberately hybrid: ML estimates future risk, while rule-based thresholds provide explainable real-time fault states.

This avoids claiming that a black-box model alone can diagnose industrial faults from a small prototype dataset.

Figure 2. Raw telemetry is converted into features, AI predictions, fault states and dashboard outputs.

7.1 Raw Data Acquisition

The Arduino samples acceleration on three axes and pressure at a fixed interval. Each packet should include timestamp or sequence number, ax, ay, az, pressure, board temperature if available and battery/power state if implemented.

7.2 Pre-processing

The edge processor applies baseline calibration, unit conversion, noise filtering and missing packet handling. A resultant vibration value can be calculated from the three accelerometer axes. Calibration is important because vibration thresholds depend heavily on mounting and test rig behaviour.

7.3 Feature Extraction

A sliding window is used to calculate RMS vibration, peak value, standard deviation, kurtosis, moving average, pressure delta and trend slope. Fault decisions should use windows instead of single readings to reduce false alarms.

7.4 AI and Analytics

Linear regression estimates short-term vibration trend/forecast. Logistic regression estimates probability of failure/risk based on features. Health scoring and trend analysis are presented as analytics modules rather than falsely labelled as separate ML models.

7.5 Fault Classification

The fault engine converts features and model outputs into three dashboard states: bearing wear risk, pressure anomaly and impact event. Each fault includes severity, confidence, timestamp, explanation and recommended action.

8. BLE Communication Design

The BLE interface should be designed as a small custom GATT service. The Arduino acts as a BLE peripheral and the Raspberry Pi acts as the central device. Telemetry should be sent using notifications rather than requiring the Pi to repeatedly poll the board.

Item	Design	Purpose
Device Name	IndustrialTwinNode	Human-readable BLE advertisement name.
Service UUID	Custom UUID	Groups all machine telemetry characteristics.
Telemetry Characteristic	Notify	Sends compact packets containing sequence number, timestamp offset, ax, ay, az and pressure.
Status Characteristic	Read/Notify	Reports sensor node health, battery/power state and firmware version.
Command Characteristic	Write	Allows the Pi to request calibration, start/stop streaming or change sampling rate.
Sampling Rate	Initial 25-50 Hz; stretch 100 Hz	Sufficient for dashboard trend and impact demo. True industrial vibration diagnosis may need higher sampling.
Packet Reliability	Sequence number and timestamp	Detects missed packets and helps the dashboard avoid false trend calculations.

9. Fault Detection Logic

The fault engine should be transparent and explainable. This is important because the system is being assessed academically and because industrial users need to understand why an alert was triggered.

Fault	Detection Logic	Dashboard Behaviour	Engineering Caveat
Bearing Wear Risk	Increasing vibration trend + elevated kurtosis > 3.5 + sustained abnormal RMS over a window	Red critical status, vibration gauge turns red, recommendation to inspect bearing/lubrication/mounting	Use the phrase “bearing wear risk/indication” unless validated against a real bearing dataset.
Pressure Anomaly: Leak/Blockage	Absolute pressure deviation from calibrated baseline > 10 hPa, or external transducer line pressure deviation beyond selected threshold	Yellow warning gauge and message to check for leaks, blockages, seals or airflow restrictions	Built-in LPS22HB is barometric. For direct leak/blockage claim, use external pressure transducer or controlled pressure chamber.
Impact Event / Transient Fault	Sudden vibration spike + high kurtosis > 4.0 + short duration	Temporary red spike marker; self-clears when readings return to normal; event stored in log	Should not trigger persistent failure alarm unless repeated impacts occur.

9.1 Alarm State Machine

State	Meaning
NORMAL	All signals within baseline range. Dashboard green.
WARNING	A non-critical threshold is exceeded or trend is deteriorating. Dashboard yellow.
CRITICAL	Sustained severe vibration or strong bearing wear indication. Dashboard red.

TRANSIENT	Short spike/impact detected. Dashboard shows temporary red spike and logs event, but clears automatically if stable.
RECOVERED	Fault condition has cleared after a defined stable period. Active alarm removed; event remains in history.

10. Digital Twin Representation

The system should define the “twin” as a data model, not only a visual dashboard. The twin object represents the current, historical and predicted health state of the machine.

Twin Component	Stored/Displayed Data
Machine Identity	machine_id, name, asset type, location, sensor node id
Current State	latest vibration, pressure, health score, active fault state
Historical State	time-series readings, extracted features, event history
Predicted State	24-hour vibration forecast, best/likely/worst scenarios, failure probability
Maintenance Intelligence	recommendations, severity, confidence, action priority
Configuration	baseline values, thresholds, sampling rate, model version

11. Dashboard Architecture

The dashboard should be treated as an operator interface, not just a graph page. It must make fault states clear, explain why the system reached that state and show what action is recommended.

Dashboard Component	Purpose
Header Status Bar	Machine name, connection status, last update time, current overall health.
Vibration Gauge	Current vibration severity; turns red during critical bearing/impact states.
Pressure Gauge	Pressure deviation from baseline; turns yellow for leak/blockage warning.
Fault Status Cards	Bearing wear, pressure anomaly and impact event shown as separate direct failures.
Live Trend Chart	Streaming vibration trend with fault markers and rolling average.
Forecast Panel	Best, most likely and worst-case vibration projection over 24 hours.
Recommendation Panel	AI/rule-generated maintenance suggestion with confidence and urgency.
Event Log	Timestamped alarms, clear events, impact transients and operator notes.
System Health Panel	BLE connection, sampling rate, packet loss, Pi CPU/temp and service state.

12. Backend API and Data Contracts

Endpoint	Responsibility
GET /api/status	Returns current machine state, health score and active faults.
GET /api/telemetry/latest	Returns latest raw and processed telemetry.
GET /api/telemetry/history?from=&to=	Returns historical readings for charts and analysis.
GET /api/forecast/24h	Returns best, likely and worst-case forecast series.
GET /api/events	Returns fault event history and clear/recovery events.
POST /api/calibrate	Starts baseline calibration routine.
GET /api/recommendations	Returns current recommendation text, urgency and explanation.
WebSocket /ws/live	Streams live telemetry, fault state changes and dashboard updates.

13. Suggested Data Model

Table / Collection	Main Fields
telemetry_readings	id, timestamp, seq_no, ax, ay, az, pressure_hpa, temperature, source_node
feature_windows	id, window_start, window_end, rms_vibration, peak_vibration, std_dev, kurtosis, trend_slope, pressure_delta
predictions	id, timestamp, forecast_horizon, best_case, likely_case, worst_case, failure_probability, model_version
machine_state	machine_id, health_score, active_fault, severity, confidence, last_update
fault_events	event_id, type, severity, started_at, cleared_at, duration, evidence, recommendation
configuration	machine_id, baseline_vibration, baseline_pressure, thresholds, sampling_rate, firmware_version

14. Raspberry Pi Deployment Architecture

Area	Architecture Decision
Operating System	Raspberry Pi OS Lite/Desktop depending on whether local browser display is required.
Python Environment	Virtual environment containing BLE collector, ML libraries, FastAPI and database client.
Services	collector.service, api.service and optional dashboard.service managed by systemd.
Networking	Local Wi-Fi/Ethernet. Dashboard accessed from Pi browser or another laptop on same network.
Logs	Service logs via journalctl plus application logs saved to file for debugging.
Startup Behaviour	On boot: start BLE collector, connect to sensor node, start API, dashboard becomes available.

15. Reliability, Safety and Security Considerations

Consideration	Control / Mitigation
Sensor mounting	Loose mounting can create false vibration readings. Use a rigid bracket and record mounting method.
Baseline drift	Pressure and vibration baselines may drift. Provide recalibration and show baseline timestamp.
BLE dropouts	Implement reconnection, packet sequence checks and dashboard connection warning.
False positives	Use sliding windows, hysteresis and transient-event classification.
False negatives	Do not claim industrial safety certification. Present as prototype risk indication.
Mechanical safety	Use guarded low-power motor/fan. Avoid exposed moving parts in demonstration.
Electrical safety	Use low-voltage DC power and stable power supplies.
Data integrity	Timestamp every reading and keep raw readings separate from derived features.
Cybersecurity	Local dashboard only for prototype; if network exposed, add authentication and HTTPS.
Model governance	Record model version, training data source and thresholds so decisions are reproducible.

16. Technical Testing Strategy

Test Area	Expected Evidence
Sensor Test	Confirm Arduino reads accelerometer and pressure values correctly; verify values change with movement/controlled pressure.
BLE Test	Confirm Pi discovers Arduino, subscribes to telemetry and receives stable packet stream.
Feature Test	Feed known sample data and verify RMS, peak, kurtosis and pressure delta calculations.
Fault Rule Test	Use simulated datasets for normal, bearing-risk, pressure anomaly and impact states.
Dashboard Test	Confirm gauges, cards, chart and recommendations update in real time.
Persistence Test	Verify readings, features and events are stored and can be retrieved.
Recovery Test	Disconnect BLE/power temporarily and verify system handles reconnection gracefully.
Demo Test	Run full physical demonstration with normal, spike, pressure anomaly and sustained vibration scenarios.

17. Implementation Phases

Phase	Deliverable
Phase 1 - Hardware Bring-Up	Arduino sensor readings over serial, confirm sensor

	availability and board revision.
Phase 2 - BLE Telemetry	Create BLE service, connect Pi collector, receive stable readings.
Phase 3 - Edge Processing	Build feature extraction pipeline and baseline calibration.
Phase 4 - Fault Engine	Implement bearing risk, pressure anomaly and impact event logic with hysteresis.
Phase 5 - ML Forecasting	Train/fit linear regression and logistic regression modules using collected/simulated labelled data.
Phase 6 - Dashboard	Build live gauges, trend chart, forecast panel, recommendation panel and event log.
Phase 7 - Integration Testing	Run complete physical demo and record evidence for presentation/report.
Phase 8 - Optimisation	Improve sampling rate, reduce latency, clean UI, document limitations and future work.

18. Key Technical Risks and Design Challenges

Risk	Severity	Mitigation
Pressure sensor credibility	High	Built-in sensor may not prove leak/blockage detection. Mitigate by controlled demo wording or external transducer.
Insufficient vibration sampling rate	Medium	Low sampling may miss useful vibration signatures. Mitigate by focusing on RMS/trend/spike rather than advanced spectral diagnosis.
Lack of labelled failure data	High	Real bearing failure data is hard to collect. Mitigate using simulated fault conditions and transparent rule-based logic.
BLE instability	Medium	Wireless dropouts can interrupt dashboard. Mitigate with reconnection and local status warning.
Overclaiming digital twin	Medium	A dashboard alone is not a full twin. Mitigate by defining a twin data model with current, historical and predicted state.
Mechanical demo safety	Medium	Moving parts can be unsafe. Use low-power motor/fan and guarded enclosure.
Model accuracy challenge	Medium	Simple regression may be limited. Present ML as risk estimation and forecast support, not safety-certified diagnosis.

19. Future Expansion Architecture

Expansion	Architecture Change Required
Rotor imbalance	Add FFT/spectral features and frequency-domain analysis based on shaft speed.
Misalignment	Combine vibration directionality, harmonics and trend features.

Mechanical looseness	Look for harmonics, unstable amplitude and repeated impact-like events.
TinyML on Arduino	Move lightweight anomaly classifier onto microcontroller for stronger embedded intelligence evidence.
Cloud sync	Send summarised events to Azure/AWS for remote monitoring while keeping inference local.
Multi-machine support	Support multiple Arduino sensor nodes and machine IDs.
Role-based dashboard	Add technician/manager views and maintenance acknowledgement workflow.
Real industrial sensors	Replace demo sensors with industrial accelerometer and pressure transducer.

20. Recommended Final Architecture for Coursework

The recommended version for assessment is a realistic, defensible prototype:

- Arduino Nano 33 BLE Sense collects vibration and pressure/environment readings.
- Raspberry Pi receives BLE telemetry and performs feature extraction, inference, event detection and dashboard serving.
- Fault detection uses hybrid logic: ML-assisted risk and forecasting plus deterministic threshold rules for explainability.
- The dashboard displays direct fault states, health score, forecast scenarios, live trend and maintenance recommendation.
- Pressure anomaly detection is documented carefully as a controlled demo unless an external pressure transducer is added.
- The final report should clearly separate implemented features, assumptions, limitations and future industrial expansion.

21. Technical References

Arduino Nano 33 BLE Sense documentation: <https://docs.arduino.cc/hardware/nano-33-ble-sense>

Arduino Nano 33 BLE Sense Rev2 documentation: <https://docs.arduino.cc/tutorials/nano-33-ble-sense-rev2/cheat-sheet>

Raspberry Pi 4 Model B product brief: <https://pip.raspberrypi.com/documents/RP-008344-DS-raspberry-pi-4-product-brief.pdf>

Raspberry Pi 4 Model B product page: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>

STMicroelectronics LSM9DS1 datasheet: <https://www.st.com/resource/en/datasheet/lsm9ds1.pdf>

Arduino Nano 33 BLE Sense Rev2 datasheet:
<https://cdn-reichelt.de/documents/datenblatt/R600/ABX00069-DATASHEET.pdf>